資訊安全實戰培訓系列

Pwn 攻防實戰培訓

從二進位基礎，到漏洞修補

適合初學者 · 繁體中文講解 · 含實作 Demo
不需要深厚的程式基礎，會一點 C 語言和 Python 就夠了

這堂課要帶大家學會什麼？

① **看得懂程式**：二進位檔案、記憶體配置、函式呼叫是怎麼運作的
② **找得出漏洞**：棧溢位、格式化字串、命令注入這些常見漏洞的成因
③ **會用工具鏈**：GDB、pwntools、checksec，組出自己的分析環境
④ **修得了漏洞**：程式碼審查、Patch 修補、驗證修補效果
⑤ **善用 AI 輔助**：知道怎麼讓 AI 幫忙，也知道為什麼不能盲信 AI

學完之後：你能獨立分析一支小程式、找出漏洞、動手修補，並驗證修補有效。

4 小時課程地圖：先懂原理，再練實作

第一部分・Pwn 攻防基礎（3 小時）

模組一　二進位基礎與程式執行原理（25 分）
模組二　ELF 結構與函式呼叫機制（30 分）
模組三　常見保護機制與利用影響（20 分）
模組四　Pwn 工具鏈與除錯環境（25 分）
模組五　常見漏洞與基礎利用思路（35 分）
模組六　綜合分析與 MIPS32 入門（25 分）

第二部分・審查與修補（1 小時）

模組七　C 原始碼審計與漏洞定位（15 分）
模組八　漏洞修補與 Patch 實作（20 分）
模組九　AI 輔助逆向分析與 Exploit 開發（25 分）

特別提醒：
第二部分包含強制 Demo，請準備可控的本機程式或合法練習題。

第一部分 · Pwn 攻防基礎（3 小時）

為什麼程式會被「攻破」？

先想一個問題：一支普通的程式，為什麼有人能讓它做「不該做的事」？
你可能會卡在這裡：明明程式是別人寫好的，輸入框也只能打幾個字，
為什麼就有人能讓它讀取秘密、執行命令、甚至拿下整台伺服器？

什麼是 Pwn？跟駭客攻防有什麼關係？

Pwn 這個詞怎麼來的？

它從遊戲術語 own（佔有、拿下）演變而來，意思就是「把程式變成你的」。

二進位安全在研究什麼？

研究「已經編譯好的程式」的安全問題——不看原始碼，直接看程式本身。

常見的攻擊目標：

· 讓程式崩潰（拒絕服務）　· 跳出權限（拿到更高權限）

· 讀取不該讀的資料（洩漏秘密）　· 執行不該執行的程式碼（遠端控制）

一句話記住：Pwn 就是跟程式「搶控制權」——讓程式聽你的話，而不是聽作者的。

一支 C 程式是怎麼「活」起來的？

① main() 是起點

程式從 main 開始跑，就像電影從第一幕開始。

② 程式的三件事

輸入 → 處理 → 輸出，幾乎所有程式都是這個循環。

③ 變數各有各的家

區域變數、全域變數、動態配置的記憶體，放的地方都不一樣。

④ 危險的源頭

程式會「相信」使用者的輸入——這正是漏洞的起點。

一支最簡單的 C 程式

```
#include <stdio.h>

int main() {
    char name[32];
    printf("請輸入名字：");
    scanf("%s", name);    <-- 輸入
    printf("你好，%s\n", name);    <-- 處理 + 輸出
    return 0;
}
```

想一想：如果輸入超過 31 個字元，
那個 name 陣列會發生什麼事？

程式跑起來時，記憶體長什麼樣？

把記憶體想像成一棟公寓大樓，每個區段住著不同「房客」：

程式碼區段
放指令（程式怎麼做）
通常是唯讀的

資料區段
放全域變數
程式啟動就分配好

堆（Heap）
執行中動態要來的空間
malloc / free 管這裡

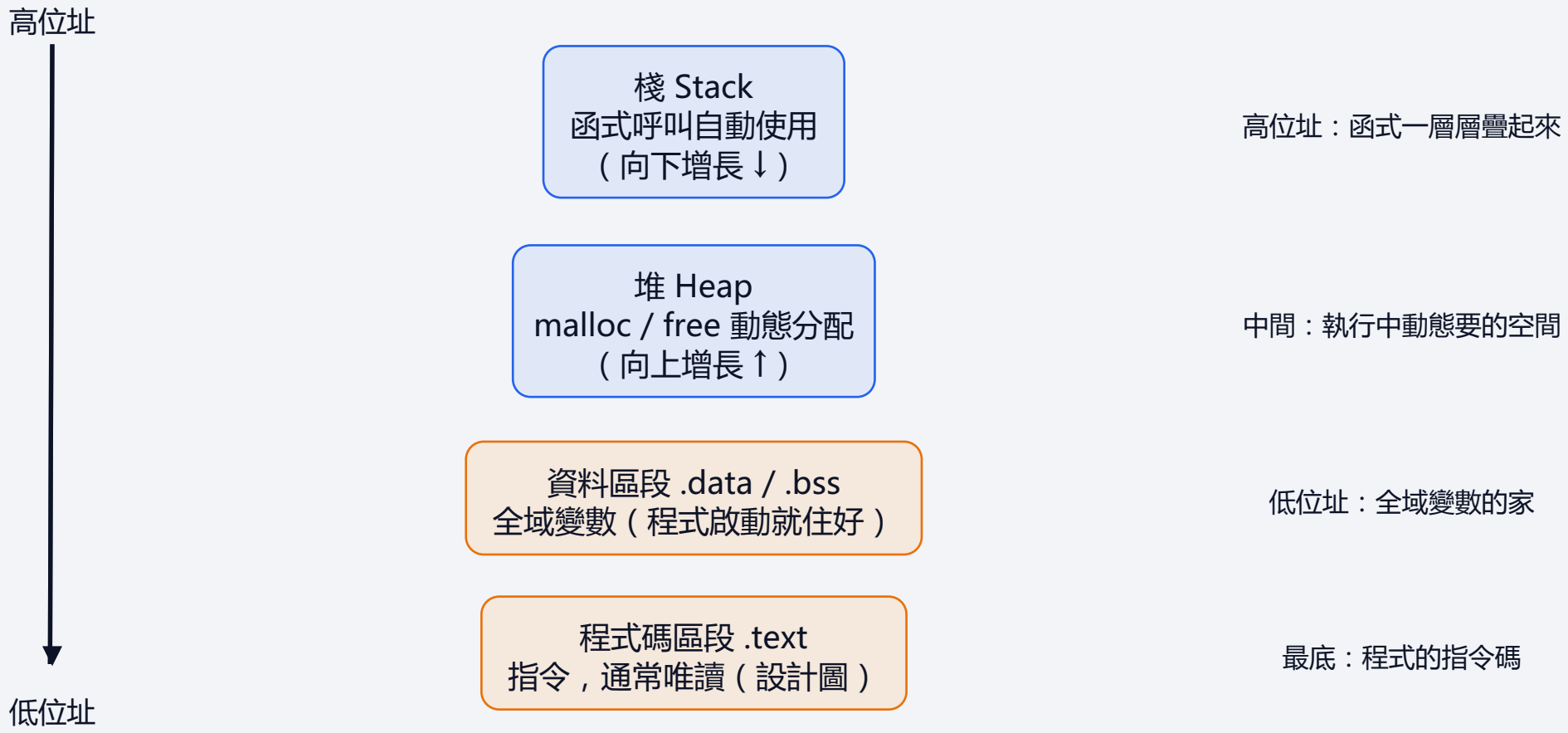
棧（Stack）
函式呼叫自動使用
等下重點講它

記住這個比喻：

· **程式碼區段** = 設計圖（只准看，不准改）
· **資料區段** = 長期住戶（程式開機就入住）
· **堆** = 臨時租房（需要時才租，退租要還）
· **棧** = 一疊盤子（後放的在上面，函式呼叫時自動疊、自動收）

記憶體長什麼樣？一張圖看懂四個區段

從高位址往低位址看，棧往下長、堆往上長，中間留給動態使用——記住這張圖，後面全靠它。



為什麼多寫幾個字，就會蓋掉別人的資料？

記憶體沒有「邊界柵欄」：寫進緩衝區的資料，一旦超過預留空間，就直接蓋到隔壁——這就是緩衝區溢位。

正常：輸入「ABCD」
剛好塞進 buf[16]，沒有越界

buf[16] 預留空間
只裝得下 16 個位元組

溢位：輸入「ABCDEFGHIJKLMNOPQRSTUVWX」
超過 buf[16] → 紅色部分蓋到隔壁

被蓋掉的可能是：另一個變數 / 返回位址

什麼是「記憶體錯誤」？程式為什麼會崩潰？

Segmentation Fault（段錯誤）
程式碰到了「不屬於它的記憶體」。最常見的崩潰原因，也是 Pwn 題的起點信號。

緩衝區溢位
寫入的資料超過了預留空間，把「隔壁」的資料蓋掉了。這不是當機而已，是漏洞。

越界讀寫（OOB）
讀或寫超出了陣列 / 緩衝區的範圍。讀越界可能洩漏秘密，寫越界可能改寫資料。

為什麼要先認識崩潰？

· 崩潰 = 程式碰到了「它不該碰的地方」——這個地方常常就是漏洞的入口
· 分析 Pwn 題的第一步，常常就是「故意打壞它」，看它崩在哪裡、崩得多慘
· 崩潰訊息會告訴你：壞在哪個位址、哪條指令、哪個函式

一句話：會崩潰的程式不可怕，可怕的是「崩潰的位置可以被控制」。

怎麼從原始碼追蹤邏輯、找到「可疑輸入」？

把程式當成「流水線」來讀：原料從哪進來、經過哪些加工、最後去哪。

① 找輸入點
scanf、gets、read、
命令列參數、網路封包

→

② 追資料流
輸入的資料去了哪？
怎麼複製、怎麼比對、怎麼使用？

→

③ 找敏感操作
system、exec、指標運算、
陣列索引、記憶體複製

→

④ 問三問
長度檢查了？
邊界算對？
輸入被當指令？

實際做法：從原始碼開始讀，一行一行追「輸入變數」的生命週期。

· 有原始碼：直接追蹤變數流向，問「這筆資料最後會碰到誰」

· 沒有原始碼：反組譯之後照樣追，工具幫你標出函式呼叫

核心問題：外部輸入有沒有可能到達「危險操作」？中間有沒有檢查？

一場 Pwn 攻防，就像一場「遠端較勁」

典型 Pwn 題目的完整流程，一共五步：



幾個重要名詞：

· **Flag**：題目藏在伺服器裡的通關字串，拿到它代表解題成功

· **Exploit（利用程式）**：把漏洞變成實際攻擊效果的腳本或程式

· **Payload**：我們送進程式裡的那段「特別輸入」

· **Shellcode**：直接執行的機器碼，通常用來開一個 shell（命令列）

ELF 檔是什麼？一支程式怎麼「打包」自己？

ELF 是 Linux 上程式的標準格式，就像一本「有目錄的書」：

ELF Header（書的封面）：告訴你這是 32 / 64 位元、什麼架構、入口在哪

.text 程式碼（指令）
.rodata 唯讀資料（字串）
.plt / .got 動態連結區

.data 已初始化的全域變數
.bss 未初始化的全域變數
.init / .fini 啟動、收尾

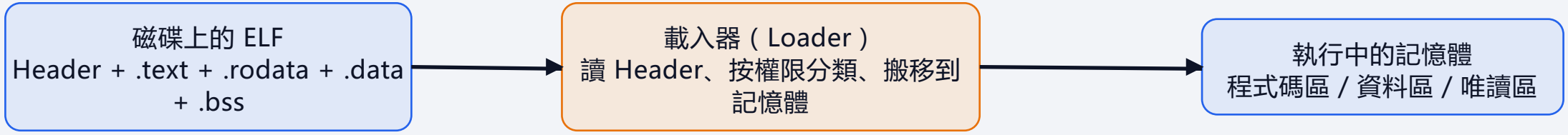
為什麼要知道這些？
攻擊和修補都要「定位」：
shellcode 放哪、GOT 在哪、
全域旗標在哪。

實用工具：

· **file**：一眼看出架構與格式（32 / 64 位元、x86 / ARM / MIPS）
· **readelf**：看 ELF 的區段、符號、動態連結資訊
· **objdump**：反組譯程式碼

ELF 怎麼從磁碟，變成執行中的程式？

磁碟上的 ELF 是一本「有目錄的書」；執行時由載入器把各區段搬到記憶體，搬到哪都有固定規則。



磁碟檔：靜態，還沒跑

執行時：系統幫你搬

記憶體：依位址排列

PLT / GOT 是什麼？程式怎麼呼叫「別人」的函式？

程式很少把 printf 寫死在自己裡面，它呼叫的是「別人家的函式」——libc。

動態連結
printf、system 這些函式放在共享程式庫 libc 裡，程式執行時才「約」過來。

PLT（跳板）
程式呼叫外部函式的「中轉站」：程式一律先跳到 PLT，再由 PLT 帶路。

GOT（地址表）
一張「電話簿」，記錄外部函式真正的位址；第一次呼叫後才填上。

為什麼這兩個區段是兵家必爭之地？

· **洩漏 GOT**：讀 GOT 可以算出 libc 的載入位址，進而算出 system 在哪
· **覆寫 GOT**：如果把 printf 的位址改成 system，程式以後呼叫 printf 就變成呼叫 system
· 所以後面講到 RELRO 保護時，就是在問「GOT 這本電話簿到底能不能改」

函式呼叫時，參數是怎麼傳遞的？

不同的 CPU 有不同的「默契」，這套規則叫呼叫約定（Calling Convention）：

項目	x86-64（64 位元）	記法
前 6 個參數	rdi、rsi、rdx、rcx、r8、r9	暫存器傳遞
返回值	rax	用 rax 帶回家
棧管理	rsp 指向棧頂、rbp 指框架	push / pop / call / ret

一個具體例子：呼叫 **printf("%s", name)** 時，系統會把
第 1 個參數（格式字串位址）放進 **rdi**，第 2 個參數（name 位址）放進 **rsi**，然後才 call。
為什麼 Pwn 要懂這個？因為寫 payload 時，要讓程式「聽話地呼叫某個函式」、
就得照規則把參數排進對的暫存器或棧上——排錯位置，攻擊就失敗。

棧（Stack）長什麼樣？為什麼「返回位址」是命門？

一次函式呼叫，會在棧上疊起一個「棧框」：



三個關鍵事實：

① 棧從高位址往低位址長，像一疊盤子，後放的在上面。

② 輸入的資料通常先落在「區域變數」，而返回位址就在它上方不遠處。

③ 如果輸入太多，會一路往上蓋：區域變數 → 舊 rbp → **返回位址**。

命門就在這裡：

蓋掉返回位址，函式結束 return 時，程式就會跳去攻擊者指定的位址執行——控制權就換人了。

蓋掉返回位址之後，程式去了哪裡？

返回位址決定「函式結束後跳去哪」。一旦被蓋成攻擊者指定的位址，return 時控制權就換人。



一張圖記住：return 跳去哪裡，程式就去哪裡。

把前面串起來：一段 C 程式，在記憶體裡長什麼樣？

原始碼

```
int global_flag = 1;   // 放 .data
char msg[] = "Hello";   // 放 .rodata

void func(char *input) {
    char buf[16];         // 區域變數 → 棧
    strcpy(buf, input); // 沒檢查長度！
}

int main() {
    char *p = malloc(32); // 動態 → 堆
    func(p);
    return 0;
}
```

執行時的記憶體對應

程式碼區段：main、func 的指令
.data：global_flag = 1
.rodata："Hello" 字串
堆：malloc 要來的 32 位元組
棧（呼叫 func 時）：
高位址 ← [返回位址] [舊 rbp] [buf[16]] → 低位址

風險點：strcpy 不檢查長度，
輸入一長，buf 就往上蓋到返回位址。

本節收穫：現在你已經能把「原始碼」和「執行時的記憶體」對起來了。
全域變數住資料區、字串住唯讀區、malloc 住堆、區域變數住棧。接下來要問：這些「家」外面有沒有上鎖？

怎麼一眼看出程式的防護等級？用 checksec

分析任何程式前，先跑 checksec——它是 Pwn 分析的「第一動作」：

```
$ checksec ./vuln
Arch: amd64-64-little    RELRO: Partial RELRO    Stack: No canary found
NX: NX enabled    PIE: PIE enabled    RPATH: RUNPATH: Symbols: 83
```

怎麼讀這張「體檢表」？

· 每一項保護機制，要嘛開（enabled）、要嘛關（disabled / no）

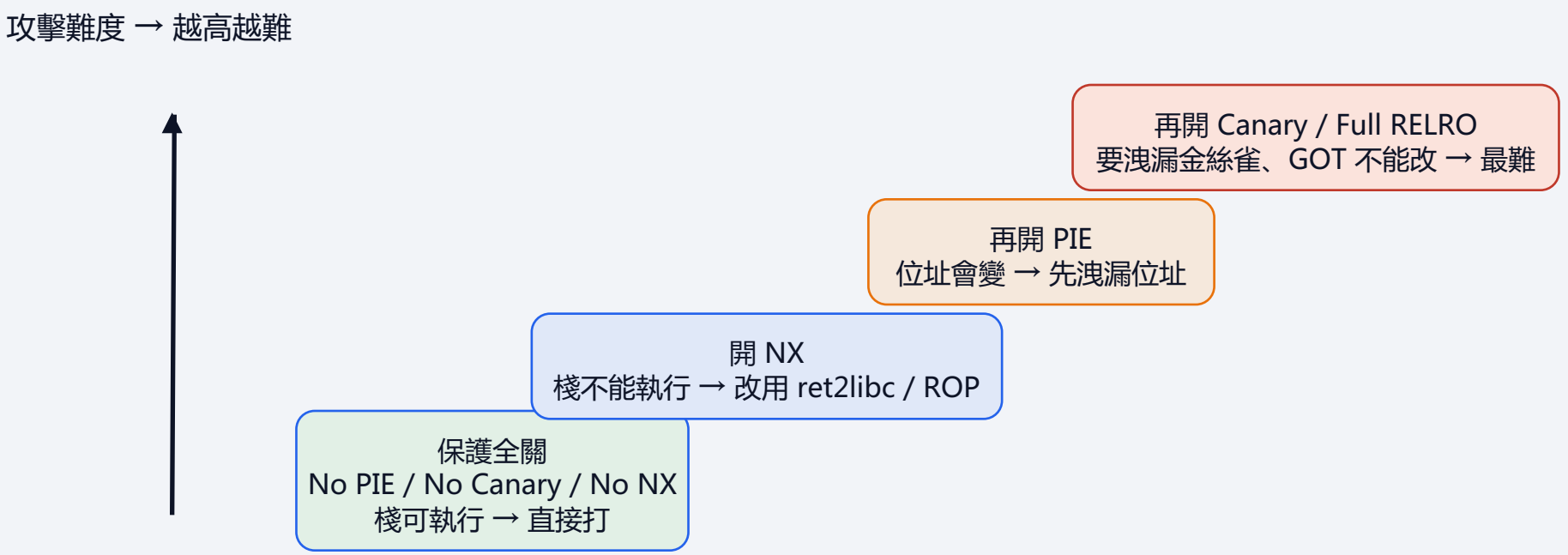
· 開關組合決定攻擊難度：全關最容易，全開最難

· 判斷順序：先看 Canary → NX → PIE → RELRO，逐項想「這題要用什麼手法」

一句話：checksec 像醫生的聽診器，先聽心跳，再決定怎麼治療。

保護機制開多開少，攻擊難度差多少？

每多開一道保護，攻擊就得多繞一關——把難度畫成階梯，一眼看清「全關」和「全開」的差別。



PIE 是什麼？為什麼程式的位址一直變？

PIE 讓程式每次載入的位址都不同，攻擊者就不能「寫死」一個位址。

沒開 PIE（固定）
程式每次都載入在同一個位址，
例如 main 固定在某個 0x40xxxx 位址，
攻擊者可以「寫死」位址直接打。

開了 PIE（隨機）
每次載入位址都不同，像搬家沒有固定門牌。
攻擊者要先「洩漏」出實際位址，
才能精準攻擊。

這帶來什麼影響？

· 沒有 PIE：返回位址可以寫死，攻擊「一發命中」
· 有 PIE：位址是「基址 + 偏移」，要先想辦法從程式裡洩漏一個位址算出基址
· 洩漏管道：格式化字串、越界讀、程式主動印出來的資料

一句話：PIE 讓「固定地址」變成「隨機地址」，攻擊多了一道「先洩漏」的關卡。

Canary（金絲雀）怎麼防棧溢位？

名字來自礦工的古老智慧：金絲雀先死，人就知道有毒氣。

它是怎麼運作的？

① 函式開頭：在「區域變數與返回位址之間」
　　放一個隨機值（canary）
② 函式結尾：檢查 canary 是否還是原值
③ 被改動了 → 立刻中止程式

攻擊者怎麼繞？

· 洩漏 canary：例如用格式化字串把 canary 讀出來
· 再原樣寫回：payload 裡先補上 canary，再蓋返回位址
· 不碰棧：改用其他漏洞（堆、格式化字串等）

重要細節：

· canary 每個執行時期都不同，且以 \x00 開頭（防止被字串函式直接讀走）
· checksec 顯示 No canary found 時，棧溢位通常可以直接蓋返回位址
· 看到 Stack: Canary found，就要開始想「怎麼洩漏或繞過」

一句話：Canary 是棧上的哨兵，讓「直接蓋返回位址」這條路走不通。

NX 和 RELRO：不能執行、不能改

NX（No eXecute）
記憶體頁面「不可執行」。
· 棧、堆、資料區只能讀寫，不能當程式碼跑
· 攻擊者放在棧上的 shellcode 直接跑不起來
· 影響：攻擊轉向 ret2libc、ROP 等「借用現成程式碼」的手法

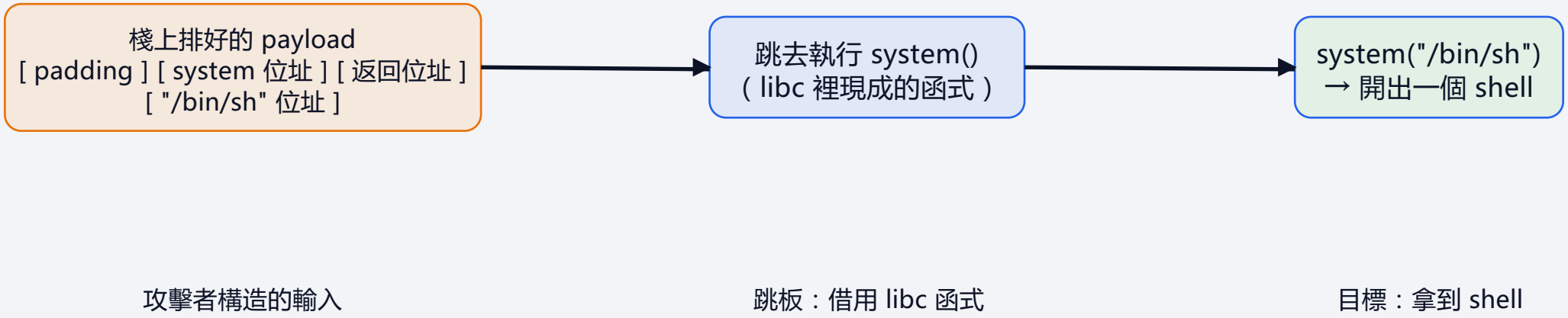
RELRO
GOT（電話簿）的唯讀保護。
· **Partial RELRO**：部分函式執行時才解析，GOT 可寫
· **Full RELRO**：全部提早解析，GOT 唯讀
· 影響：Full RELRO 下，「覆寫 GOT 偷換函式」這招失效

兩種保護一起看，就是「攻防貓抓老鼠」：

· 不讓你在棧上執行 → 那就跳去 libc 裡執行（ret2libc）
· 不讓你改 GOT → 那就改別的地方，或直接洩漏位址
· 每關一道門，攻擊者就繞一道窗——所以「組合」才是重點

NX 開啟時，攻擊怎麼「借用」libc？

棧上不能執行，那就「打不過就加入」：讓 return 跳去執行 libc 裡現成的 system()。



同樣一個漏洞，保護機制不同，攻擊差很多

把四種保護放進一張表，一次看懂「開關的影響」：

保護機制	關閉時（攻擊輕鬆）	開啟時（攻擊要繞）
PIE	位址固定，直接寫死返回位址	要先洩漏位址、算出基址
Canary	直接蓋返回位址	先洩漏 canary 或繞過棧
NX	棧上直接執行 shellcode	改用 ret2libc、ROP
RELRO	可覆寫 GOT 偷換函式	不能改 GOT，換攻擊目標

實戰中的讀表方法：拿到一支程式，先 checksec，
然後從「最弱」的保護開始想：哪裡沒開，就往哪裡打。
例如「NX enabled + No canary + No PIE」= 直接 ret2libc，還不用洩漏位址。
練習目標：看到 checksec 輸出，能在 10 秒內說出「大概該用什麼手法」。

GDB 怎麼用？除錯器的四個基本動作

GDB 是 Linux 上的除錯器，Pwn 分析就是「用 GDB 偷看程式內部」：

四個基本動作

· **break**：設斷點，例如 break main
· **next / step**：單步執行（跳過或進入函式）
· **info registers**：看暫存器（rdi、rsp…）
· **x/20gx 位址**：看記憶體內容

典型除錯流程

gdb ./vuln
→ break main　← 停起點
→ run　← 跑到斷點
→ 輸入資料 → 單步走 → 看暫存器
→ 觀察輸入怎麼影響記憶體

為什麼 Pwn 一定要會 GDB？

· 確認漏洞：實際跑一次，看輸入是否真的蓋到了返回位址
· 算偏移：用「特殊輸入」找出蓋到返回位址需要幾個位元組
· 驗證假設：AI 或反組譯給的結論，最後都要在 GDB 裡眼見為憑

一次完整分析，工具怎麼接力合作？

三個工具不是各自為政，而是一條接力鏈：先看防護，再進 GDB 確認漏洞，最後用 pwntools 打出去。



pwndbg / peda：給 GDB 裝上「Pwn 專用外掛」

原廠 GDB 介面很陽春，裝上外掛之後，畫面立刻變成「儀表板」：

pwndbg（目前最主流）
· 自動顯示：反組譯、暫存器、棧、堆
· 常用：checksec、cyclic、pattern、find、got
· 對初學者最友善，推薦直接用

peda（另一套常用）
· 也提供自動化的暫存器與棧檢視
· 經典指令 pattern 找偏移非常順手
· 兩套裝一套即可，不要混用

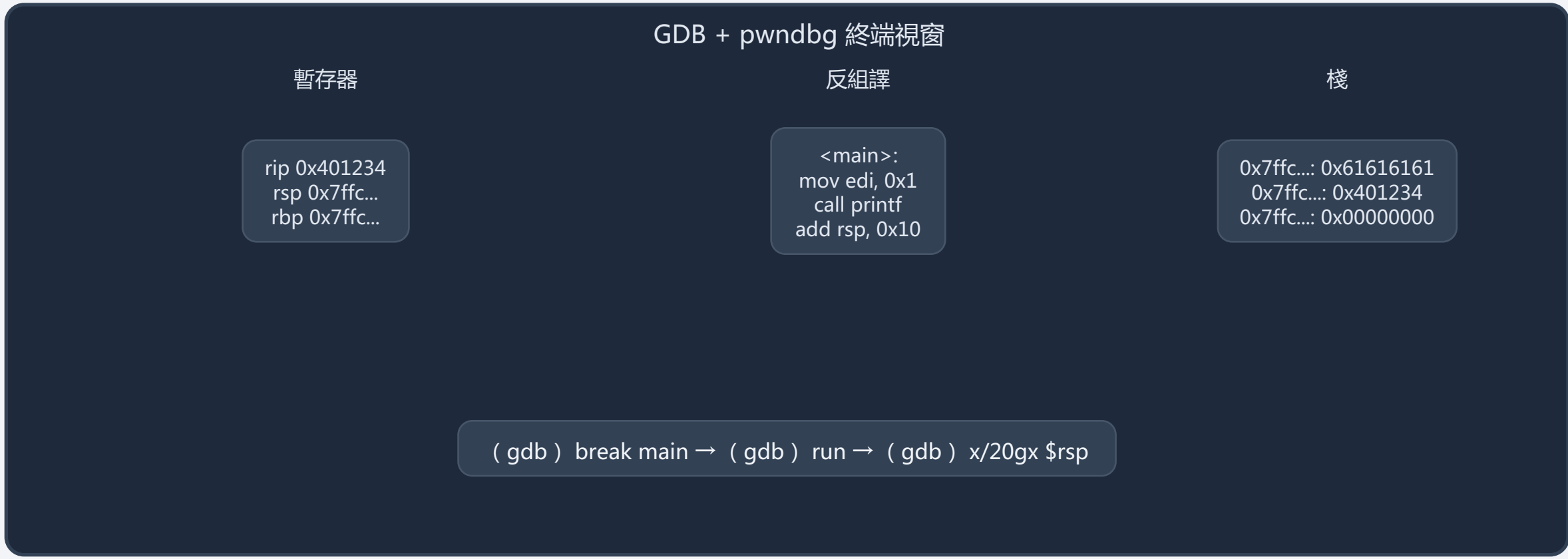
最重要的兩個「外掛指令」：

· **cyclic 200**：送一串有規律的輸入（例如 aaaabaaacaaa...）
· **pattern search**：崩潰後輸入它，立刻算出「蓋到返回位址要幾個位元組」

一句話：外掛把 GDB 從「文字介面」變成「Pwn 儀表板」，省下的時間非常可觀。

GDB 的畫面，四個區塊各看什麼？

裝上 pwndbg 後，GDB 從文字介面變成「儀表板」——一眼同時看到命令、暫存器、反組譯和棧。



pwntools 入門：用 Python 跟程式「對話」

pwntools 是 Python 寫 exploit 的標準工具箱，三步就能跟程式互動：

① 連上程式
process("./vuln")
remote("主機", 連接埠)
本機測試用前者，打題目用後者

② 送資料
sendline(b"payload")
把 payload 送進程式，
就像你在鍵盤打字

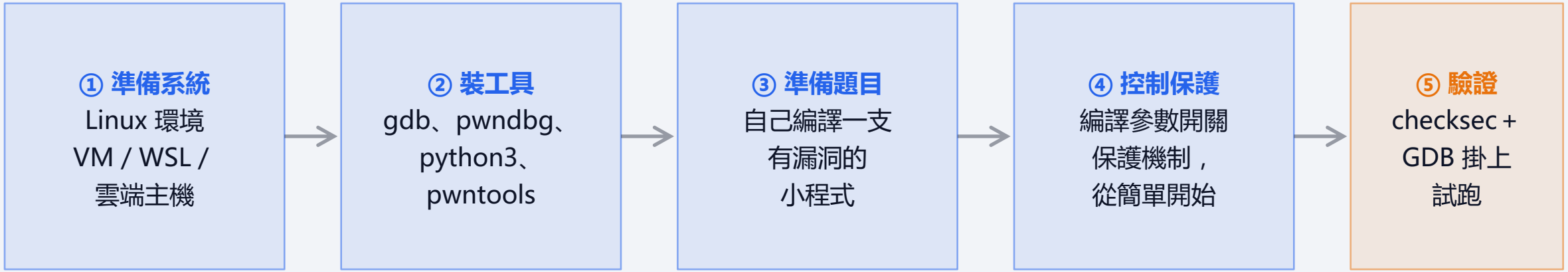
③ 收資料
recv() / recvline()
讀程式輸出的訊息，
例如洩漏出來的位址、flag

其他常用工具（用到再查）：

· **p32() / p64()**：把數字打包成位元組（地址轉成 payload 的一部分）
· **ELF()**：解析程式檔，直接查符號、區段、GOT
· **cyclic()**：產生找偏移用的規律字串
· **interactive()**：把互動權交回你的鍵盤（拿到 shell 後用）

本機練習環境怎麼搭？五步搞定

建立一套能自己練習的環境，只需要五步：



編譯參數範例：

· 全關保護（練基本功）：gcc -o vuln vuln.c **-fno-stack-protector -no-pie -z execstack**

· 開部分保護（練繞過）：gcc -o vuln vuln.c **-fstack-protector** 等

· 從「全關」開始練，一項一項把保護打開，攻擊難度跟著升級

綜合示範：用 GDB + Python 分析一支小程式

把模組四的工具串成一個完整流程，現場走一遍：

① 偵查
checksec ./vuln
→ 沒 Canary、沒 PIE
→ 初步判斷：棧溢位可直打

→

② 找偏移
GDB 裡 cyclic 200
→ 崩潰 → pattern search
→ 算出偏移（例如 40）

→

③ 打 payload
pwntools 組 payload：
padding + 目標位址
→ 連上程式 → 送出去

現場會實際示範的操作：

· 用 pwndbg 逐步看「輸入前後棧的變化」——眼見為憑，比任何說明都清楚
· 用 pwntools 送第一版 payload，看程式「如預期崩潰」在哪裡
· 修正 payload（位址、偏移），直到程式照我們的意思跳轉

這一段請跟著動手：同樣的指令自己敲一遍，工具只有「用過」才會。

棧溢位（Stack Overflow）：為什麼多打幾個字就崩了？

用杯子裝水的比喻：杯子只有 32 毫升，卻一直倒水，水就會滿出來淹到桌上。

漏洞成因

複製資料時沒有檢查長度，
把棧上「不該動的」蓋掉了。
經典危險函式：
gets()、strcpy()、sprintf() 都不檢查長度

影響

· 輕則：程式崩潰（Segmentation Fault）
· 重則：蓋掉返回位址 → 控制程式流程
· 配合 NX 關閉：直接在棧上放 shellcode 執行

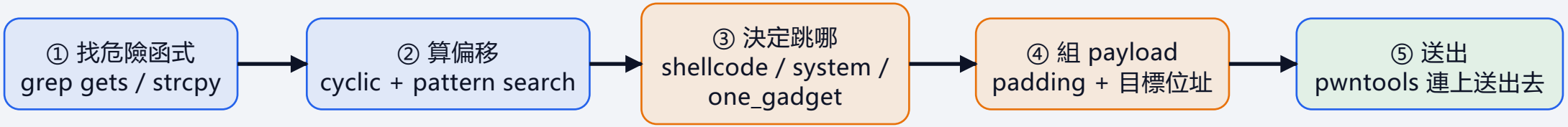
解題四步：

① 找危險函式（gets / strcpy）→ ② 用 cyclic 找偏移（蓋到返回位址要幾位元組）
③ 決定要跳去哪（shellcode / system / one_gadget）→ ④ pwntools 組 payload 送出

一句話：棧溢位是「邊界沒檢查」最典型的代表，也是所有 Pwn 手法的入門磚。

從漏洞到拿到控制權，一共幾步？

棧溢位不是一鍵搞定，而是可重複的五步流程——每一步都有明確的工具和輸出。



每一步輸出都是下一步的輸入：函式 → 偏移 → 目標 → payload → 連線

ret2libc：不能執行 shellcode，那就「借用」現成函式

NX 開啟時，棧上不能執行。但 libc 裡住著現成的 system()——那就跳去用它。

核心思路
把返回位址改成 libc 中
函式的位址（例如 system），
讓程式「幫我們」執行命令。

關鍵佈局
棧上排好：
[padding][system 位址][返回位址
]["/bin/sh" 位址]
讓 system("/bin/sh") 被呼叫

最大難點
要知道 system 和
"/bin/sh" 的實際位址。
→ 需要洩漏 libc 位址，
或用固定偏移推算

位址怎麼來？

· 程式先洩漏一個 libc 函式的位址（格式化字串、程式印出等）
· libc 內部偏移是固定的：system = 洩漏值 - 該函式偏移 + system 偏移
· 工具 one_gadget 也可以直接給「一條命令執行」的位址

一句話：ret2libc 是「打不過就加入」——跳去執行 libc 現成的函式。

格式化字串漏洞：printf 怎麼變成「任意讀寫」？

當程式把「使用者輸入」直接當成格式字串，% 符號就變成攻擊武器。

成因

printf(buf) 而不是 printf("%s", buf)
輸入裡的 %x、%s、%n 會被當成
真正的格式指令執行

洩漏（讀）

%x、%p：把棧上的東西印出來
→ 洩漏 canary、libc 位址、
返回位址

覆寫（寫）

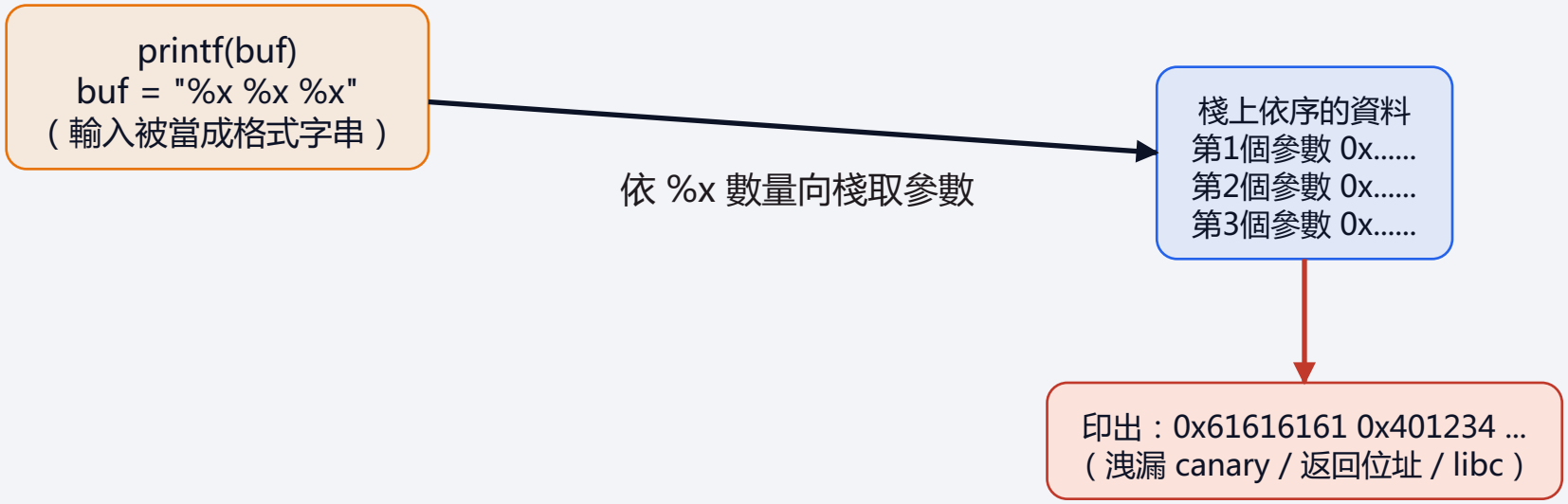
%n：把「已輸出字元數」
寫進指定位址
→ 改 GOT、改旗標、改返回位址

一句話理解：你輸入的 % 符號，讓程式「替你把記憶體印出來」或「替你寫進記憶體」。

實戰用途：常常不是單獨一種攻擊，而是「輔助技能」——用它洩漏 canary 和 libc 位址，再配合棧溢位打 ret2libc。會了格式化字串，很多「洩漏」問題都迎刃而解。

%x 是怎麼把棧上的資料「印」出來的？

當輸入被當成格式字串，%x 會讓 printf 從「下一個參數位址」開始，把棧上的資料當十六進位印出來。



命令注入：你的輸入，變成了系統命令

程式把使用者輸入「拼」進系統命令，輸入裡的符號就變成了新命令。

典型例子

程式：system("ping " + 你的輸入)
輸入：; id
結果：先 ping，再偷偷執行 id
輸入：; cat /etc/passwd → 偷讀檔案

怎麼辨識

看到 system()、popen()、exec 開頭、
shell 指令拼接，就要高度警覺：
輸入的資料有沒有可能
「逃出」字串變成命令？

危害等級：最高。可以執行任意系統命令、讀檔案、甚至直接拿權限。

一句話：命令注入的根源，是把「資料」當成了「指令」——輸入沒有被隔離。

它跟格式化字串、棧溢位其實是同一家族：**都信任了外部輸入。**

三種漏洞，一個共同點：沒檢查邊界

漏洞	信任了什麼	後果
棧溢位	輸入的長度不會超過緩衝區	蓋掉返回位址，控制流程
格式化字串	輸入只是普通文字	洩漏 / 改寫記憶體
命令注入	輸入只是字串資料	執行任意命令
共同根源	外部輸入沒有被「驗證」和「隔離」——程式過度信任輸入	

這個認知很重要：學會一種漏洞，等於學會了一族的思維。

往後看到任何程式，都先問：**輸入從哪來？被當成什麼用了？**

· 當成資料寫進記憶體 → 檢查長度和邊界

· 當成格式 / 指令 → 檢查有沒有被「執行」的可能

把洞補上：三個通用的修補原則

攻擊手法千變萬化，修補原則其實只有三個：

① 檢查長度
不用 gets、strcpy、sprintf，
改用 fgets、strncpy、snprintf，
並且算清楚目標空間夠不夠

② 不要信任輸入
格式字串固定寫死：
printf("%s", buf)；
命令用參數化，不拼接

③ 最小權限
程式用最小權限跑，
就算被攻破，
破壞力也降到最低

先想「輸入從哪來」，再想「怎麼防」：

· 輸入會進記憶體 → 管好長度和邊界
· 輸入會被格式化 / 執行 → 管好「它能不能變成指令」
· 就算都防住了，權限也要最小——這是防守的最後一道保險

預告：這三個原則，就是最後一小時「Patch 實作」的理論基礎。

MIPS32 是什麼？跟 x86 哪裡不同？

MIPS 是另一種 CPU 架構，路由器、嵌入式設備、IoT 裝置非常常見。

在哪裡出現？
路由器（如家用 AP）、
機上盒、IoT 裝置、
一些嵌入式系統

為什麼要學？
真實世界的漏洞，
很多都發生在這些設備上；
學會跨架構分析，
分析能力才算完整

初步印象
暫存器用 $ 命名：
$a0-$a3 傳參數、
$v0 返回值、$ra 存返回位址

核心差異先記三件事：

· 傳參數的暫存器不同（$a0-$a3 而不是 rdi 那組）
· 返回位址放在暫存器 $ra，而不是壓在棧上
· 指令風格不同（lw / sw 載入儲存、jal / jr 呼叫返回）

MIPS 的函式呼叫，跟 x86 差在哪？

最大的差別：MIPS 的返回位址不壓棧，而是放在暫存器 $ra。

參數傳遞
前 4 個參數放 $a0-$a3，
更多參數才放棧上。
返回值放 $v0。

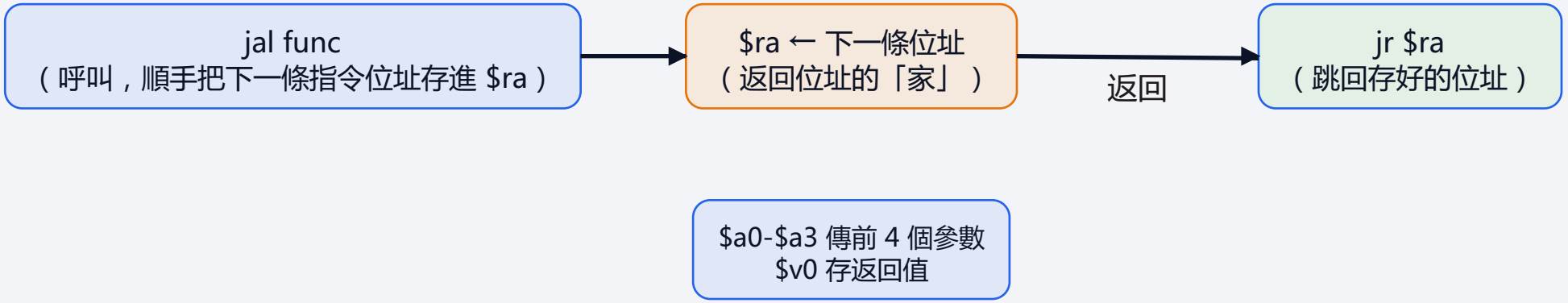
呼叫與返回
jal func：呼叫函式，
同時把「下一條指令位址」
存進 $ra。
返回用 jr $ra。

對分析的影響
在 MIPS 上找「控制流程」，
要看 $ra 被誰寫了，
而不是只看棧上的返回位址。

一句話總結：同樣的漏洞概念（棧溢位、格式化字串）在 MIPS 上依然存在，
只是「落點」不同——攻擊要改寫的目標、找位址的方式都要跟著架構調整。
學會這個思維，換任何架構都不怕：**先搞懂呼叫規則，再談利用。**

MIPS 的函式呼叫，一張圖看懂 $ra 去哪了？

MIPS 不把返回位址壓棧，而是存在暫存器 $ra——攻擊落點和 x86 完全不同。



x86 與 MIPS：一張表看懂差異

項目	x86-64	MIPS32
參數	rdi、rsi、rdx、rcx、r8、r9（6 個）	$a0-$a3（4 個）
返回值	rax	$v0
返回位址	壓在棧上	$ra 暫存器
常用指令	push、pop、call、ret	lw、sw、jal、jr

怎麼用這張表？

· 拿到一支陌生架構的程式，第一件事：查它的呼叫約定（file 指令會告訴你架構）

· 反組譯時對照這張表，就知道「參數在哪、返回值在哪、返回位址在哪」

· 分析思維不變：**找輸入 → 追資料流 → 找敏感操作 → 想呼叫規則**

一套通用的 Pwn 解題流程，走到哪都適用

把前三小時學到的東西，收斂成一套「六步解題流程」：



兩個提醒：

· 順序可以跳，但「偵查→分析→利用」不能省——直接打 payload 是新手最常踩的坑

· 每個假設都要驗證：位址算錯、偏移差一，失敗了不是猜，是回去用 GDB 量

· 這套流程在 x86、MIPS、ARM 上都適用，架構只是換「呼叫規則」而已

第一部分 · 小結

回到開頭：程式為什麼會被攻破？

答案是一條鏈：
程式會信任輸入 → 輸入沒被檢查 → 邊界被突破 → 控制權被搶走

四個關鍵詞：輸入、邊界、記憶體、控制權
兩個補充認知：保護機制不是萬靈丹，只是提高攻擊難度；
分析思維跨架構通用，x86、MIPS、ARM 都一樣。

第二部分 · Code Review 與 Patch 實作（1 小時）

發現漏洞之後，該怎麼修？AI 又能幫多少忙？

你可能會卡在這裡：漏洞找到了，直接改一行就好？改了會不會壞掉？
這一個小時，你會學到：怎麼系統性地審查程式碼、怎麼把洞補上、
怎麼驗證修補有效——還有：AI 給的分析和 exploit，你敢直接信嗎？

代碼審計是什麼？像「體檢」一樣檢查一支程式

審計 = 不執行程式，直接讀程式碼找問題（靜態分析），像醫生做體檢。

審計的切入點（三步）

· 找輸入來源：外部資料從哪進來
· 追資料處理流程：輸入怎麼被用
· 找敏感操作：system、記憶體複製、指標、權限

重點看四類問題

· 越界讀寫（緩衝區、陣列索引）
· 格式化字串不當使用
· 權限問題（高權限服務、權限暴露）
· 命令注入（system 拼接）

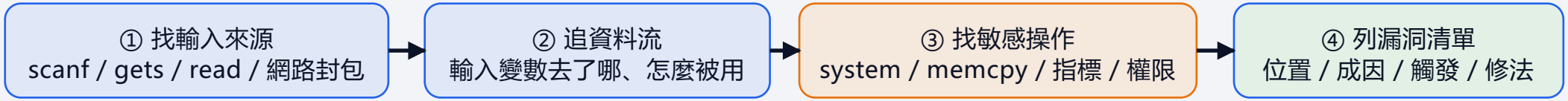
工具輔助：

· grep 搜關鍵字：搜 gets、strcpy、system、sprintf 這些「危險函式」
· 靜態分析工具：自動標出可疑的資料流
· AI 輔助：貼程式碼讓 AI 先讀一遍（後面模組九會詳細講怎麼用）

一句話：審計的目標不是看懂全部程式，而是「把可疑的地方全部標出來」。

審計一支程式，照這張圖走就不會漏

代碼審計不是漫無目的地讀，而是沿著「輸入 → 處理 → 敏感操作」這條線，把可疑點全部標出來。



一張圖記住：找入口 → 沿路追 → 看出口 → 寫清單

怎麼在程式碼裡找出「越界」的風險？

四種檢查動作，把越界問題「掃」出來：

① 找複製類函式
grep 搜 strcpy、sprintf、memcpy、gets——這些是越界的頭號嫌疑犯

② 檢查長度
複製前有沒有算目標空間夠不夠？來源資料有沒有長度上限？

③ 檢查索引
陣列索引有沒有可能超出範圍？索引值是不是使用者可控？

④ 檢查迴圈與邊界計算

· 迴圈次數受不受輸入控制？結束條件有沒有可能永遠不成立？

· 邊界計算「off-by-one」：陣列大小 16，卻允許寫到第 16 個位置？

範例判斷：看到 char buf[16]; strcpy(buf, input); 就是「複製沒檢查長度」的典型。

還有兩個高頻雷區：格式化字串、權限

越界之外，審計還要盯兩個地方：格式化字串用錯、權限給太多。

格式化字串審查

· 雷點：printf(buf)、sprintf(buf, user_input)
· 正確寫法：printf("%s", buf)、sprintf(buf, "%s", input)
· 判斷標準：**格式字串是不是寫死的？**

權限審查

· 服務是不是用 root 在跑？
· 有沒有不必要的權限（setuid、777 權限的檔案）？
· 敏感操作（讀檔、執行、寫入）有沒有做身分檢查？

審計的輸出是一張「漏洞清單」：

每條記錄四件事：**位置**（哪支檔案哪一行）、**成因**（哪裡沒檢查）、**觸發條件**（怎麼輸入會打中）、**建議修法**（改哪一行）。

清單品質決定修補品質——**找不到的清單，後面無從修起。**

示範：從輸入到敏感操作，十分鐘定位漏洞

一段簡單的 C 程式，藏著多個可疑點——我們現場走一遍審計：

```
int main(int argc, char **argv) {
    char cmd[128];
    char buf[32];
    scanf("%s", buf);    // ① 沒限長度
    sprintf(cmd, "echo %s", buf); // ② 拼接
    printf(cmd);    // ③ 格式化字串
    system(cmd);    // ④ 命令執行
    return 0;
}
```

審計發現（漏洞清單）：

① scanf 沒限制 → 棧溢位風險
② 輸入拼進命令 → 命令注入
③ 輸入當格式 → 格式化字串
④ system 執行 → 放大前三個

一句話：這支程式把「信任輸入」犯到了極致，四個敏感點環環相扣。

Demo 示範流程：grep 關鍵字 → 逐行追資料流 → 填寫漏洞清單（位置 / 成因 / 觸發 / 修法）
→ 下一步：把這張清單變成 Patch（模組八的內容）。

Patch 是什麼？修補前後差在哪？

Patch 就是對程式碼做「最小改動」，把漏洞修掉，同時不破壞原有功能。

修補三件套

① 改程式碼（最小改動）
② 重新編譯
③ 測試驗證（攻不進 + 功能正常）

好 Patch 的標準

· 漏洞沒了（再打一次打不進去）
· 功能沒壞（正常輸入照常運作）
· 只改必要處（不動無關程式碼）

別搞混

Patch 是「修原始碼」。
有人會在二進位上直接改指令（binary patching），難度更高，今天不講。

記住工作順序：重現漏洞 → 修補 → 重新編譯 → 回歸測試。

修補前先「確認漏洞能被觸發」，修補後再「確認打不進去了」——兩次測試缺一不可。

一句話：好的 Patch 像微創手術，只動該動的地方，病人（程式）還活得好好的。

同樣一段程式，修前修後差在哪？

Patch 的「最小改動」常常就是把不檢查長度的函式，換成會檢查長度的安全版本。

修補前（危險）

```
gets(buf);
strcpy(dst, src);
scanf("%s", buf);
```

不檢查長度 → 輸入一長就溢位

Patch：換函式

修補後（安全）

```
fgets(buf, size, stdin);
strncpy(dst, src, n);
scanf("%31s", buf);
```

帶長度上限 → 溢出被擋住

怎麼修棧溢位？把「不檢查長度」改成「檢查長度」

原則不變：限制輸入長度。常見的「換函式」做法：

危險寫法	安全替代	注意事項
gets(buf)	fgets(buf, size, stdin)	size 填緩衝區大小
strcpy(dst, src)	strncpy(dst, src, n)	記得自己補 \0
sprintf(buf, ...)	snprintf(buf, size, ...)	size 給正確上限
scanf("%s", buf)	scanf("%31s", buf)	格式裡寫寬度上限

陷阱提醒：「換函式」不是照抄就好——要算對緩衝區大小、要處理結尾字元。
修完不是結束，**重新編譯 + 測試**才算數（下一頁講怎麼驗證）。

格式化字串、權限、命令注入怎麼修？

三個雷區，各自對應一種修法——共同核心是「輸入不再當成程式」：

格式化字串
printf(buf)
→ printf("%s", buf)
格式字串永遠寫死，
輸入只是「資料」

命令注入
system("ping " + 輸入)
→ exec 系列陣列參數，
不經過 shell 解讀
或白名單驗證輸入

權限
服務不要用 root 跑，
檔案權限降到
「剛好可用」；
敏感操作加身分檢查

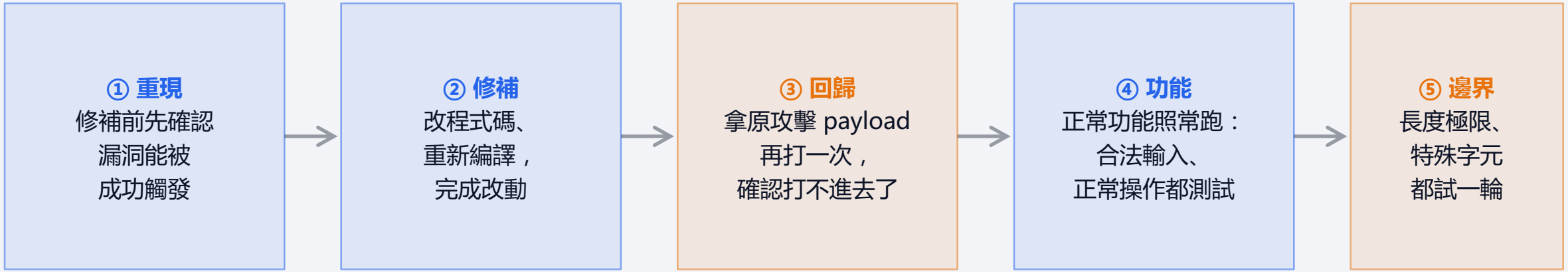
一句話總結三種修法：

讓輸入「不再是程式碼、不再是格式、不再是命令」，只是純粹的資料。

驗證方式：拿修補前的攻擊 payload 再打一次——打不進去，才算真的修好。

修完怎麼知道真的修好了？測試五步

修補不是「改完就結束」，驗證修補效果才是真正的完成：



Demo 會展示：修補前後程式碼差異、重新編譯、回歸測試三件事。

· 修補前：payload 打進去，拿到 shell / 崩潰 → 記錄證據

· 修補後：同一個 payload 再打，程式正常處理、拒絕輸入 → 對照證明

· 同時檢查：合法功能沒壞、相容性影響（例如舊設定檔仍可用）

AI 怎麼幫你看「看不懂」的反編譯代碼？

反編譯把二進位還原成接近 C 的程式碼，而 AI 擅長「讀」這種程式碼。

AI 能做什麼

· 解釋函式邏輯：「這個函式在做什麼」
· 標出資料流：輸入從哪來、往哪去
· 指出可疑操作：strcpy、system、指標運算

怎麼問（Prompt 範例）

「這是一段反編譯的 C 程式碼，請解釋：
① 每個函式做什麼　② 輸入從哪裡來
③ 哪裡有安全風險」

使用要訣：

· 貼「完整函式」而不是片段——AI 需要上下文才能判斷
· 問題要具體：「這個函式做什麼」比「有沒有漏洞」更有用
· 反編譯名字常是亂的（sub_401000），AI 的解釋要對照反組譯自己驗證

最重要的提醒：AI 看不懂的地方會「猜」，猜錯時語氣依然很肯定——結論不能直接當事實。

AI 給的結論，要過幾關才能信？

AI 說「有漏洞、該這樣打」，只是第一步——它必須過三關，才配得上「被驗證過的結論」。



三關都過，AI 的建議才變成「你的結論」——這是安全從業者的職業習慣

讓 AI 幫你整理：成因、觸發條件、修補建議

給 AI 足夠素材，要求它輸出「結構化的分析結論」，比零散問答有用得多。

給 AI 的素材

· 反編譯程式碼（完整函式）
· 程式行為：輸入什麼會出錯
· 錯誤現象：崩潰訊息、異常輸出

要求輸出格式

① 漏洞成因（一句話）
② 觸發條件（怎麼輸入）
③ 影響（能造成什麼）
④ 修補建議（改哪裡）

Prompt 範例：

「這段代碼讀取輸入後直接 strcpy 到固定陣列，請分析：漏洞成因、觸發條件、可能影響、修補建議。」

驗證鐵律：AI 說的不算，GDB 實測才算。AI 給的「觸發條件」，一定要實際打一次確認。

讓 AI 寫 exploit 初稿？可以，但要「餵對料」

AI 能生成腳本初稿，但品質取決於你給的資訊——資訊越完整，初稿越接近能用。

必須提供的資訊

· 程式結構：輸入格式、緩衝區大小
· 保護機制：checksec 輸出（關鍵！）
· 漏洞位置與偏移
· 錯誤現象：崩潰訊息

為什麼只是「初稿」？

· AI 不知道你的環境細節（libc 版本、位址、遠端服務狀態）
· 常常「差一點」：偏移錯、位址錯
· 所以：先人工檢查 → 再修改 → 再測

範例 Prompt：「程式是 64 位元、NX 開著、沒 Canary、沒 PIE，棧溢位偏移 40，輸入會印出洩漏位址。請用 pwntools 寫 ret2libc 的 exploit 初稿。」

記住：AI 產出的是「草稿」，交給 GDB 驗證之前，它只是建議。

為什麼不能直接信 AI 的分析？

AI 會犯的錯，不是「可能犯」，而是「一定會在某些時候犯」：

AI 的典型錯誤

· 位址算錯（偏移差一、基址算錯）
· 假設錯誤（沒注意保護機制）
· 過度自信（把猜測寫成事實）

怎麼驗證

· GDB 逐步核對每個位址、每個偏移
· payload 實測：本地先跑，通了再打遠端
· 交叉檢查：換個工具或自己推一遍

安全提醒

AI 給的程式碼，
執行前要先看懂：
它可能包含意料之外的
行為或惡意內容。

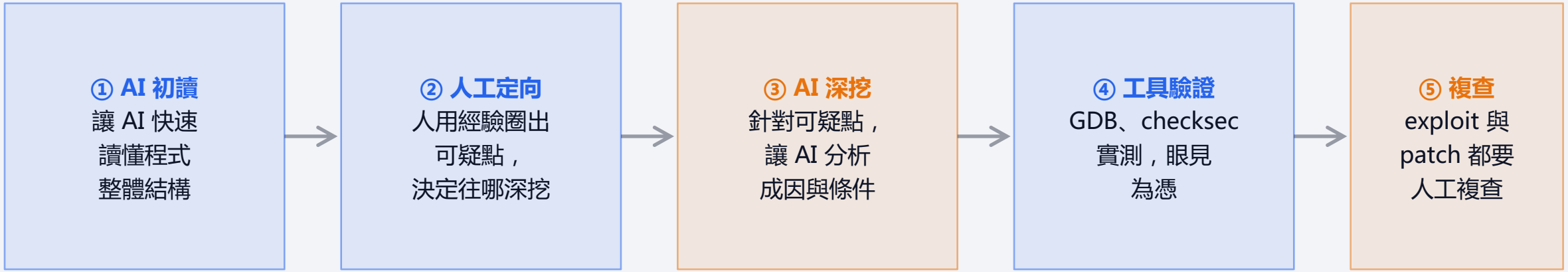
一句話總結：

AI 是助手，不是權威；**驗證是永遠的責任。**

所有 AI 給的位址、偏移、成因、腳本，都要在 GDB 裡眼見為憑、在本地實測通過，才算「被驗證過的結論」。這也是安全從業者最重要的職業習慣。

人機協作的正確姿勢：一個閉環

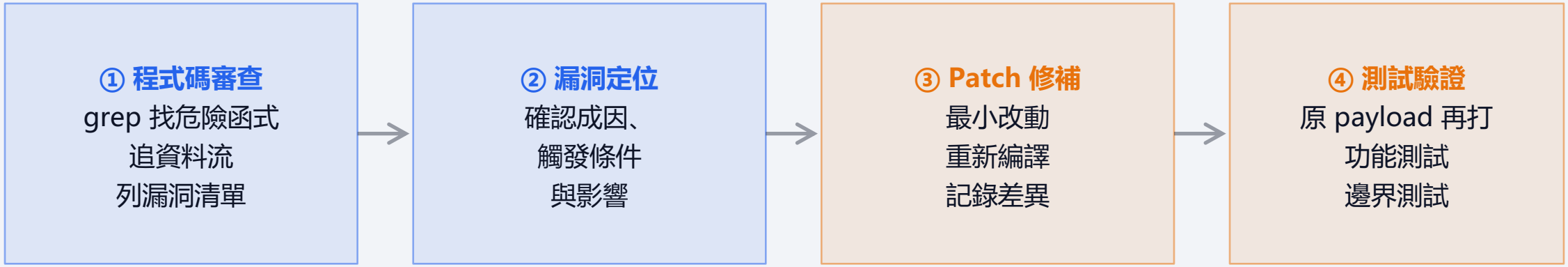
AI 不是代替你分析，而是「加速你的分析」——正確流程是一個五步閉環：



這個閉環的關鍵：每一步之間都有「人工把關」——方向由人定，
結論由工具驗，AI 負責的是「快」：快速讀懂、快速整理、快速出草稿。
效率正確配比：AI 幫你省 80% 的「讀」的時間，剩下的 20%「驗證」，
必須自己完成——這 20% 才是安全從業者的價值所在。

實作 Demo：完整展示「審查 → 修補 → 驗證」

課程要求的 Demo，就是要大家親手走完這條流程：



加分項：展示 AI 輔助分析 + 人工除錯的實際操作——
用 AI 讀代碼、出分析，再用 GDB 驗證 AI 的結論對不對。
合法要求：只能用可控的本機測試程式，或合法的 Pwn 練習題（例如 CTF 平台的練習題），**不得使用任何未授權的系統。**

四個小時，你應該帶走什麼？

第一部分重點（找漏洞）
· 信任輸入是萬惡之源
· 會讀組合語言：看懂呼叫、看懂棧
· 知道保護機制在想什麼

第二部分重點（修漏洞）
· 審計 = 把可疑處標出來
· Patch = 最小改動 + 重新編譯 + 驗證
· AI 是助手，驗證是責任

課程強調的三件事：

① 所有結論都要**眼見為憑**（GDB、實測）　② 所有操作都要**合法合規**　③ 學會**自己動手**，不依賴 AI

驗收標準：能獨立完成「審查 → 定位 → 修補 → 驗證」整條流程的 Demo，
就代表四個小時沒有白費。

第二部分 · 小結

找到漏洞，更要修得漂亮

這一個小時，我們學會了：

審計三步：找輸入、追流程、查敏感操作

修補三件事：改代碼、重新編譯、測試驗證

AI 協作五步閉環：初讀、定向、深挖、驗證、複查

一條鐵律貫穿始終：AI 是助手，不是權威；驗證是永遠的責任。

從信任輸入，到守住邊界

Pwn 攻防實戰培訓 · 謝謝大家

歡迎提問與交流 · 請完成實作 Demo · 學以致用、合法合規